



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

VICERRECTORADO ACADÉMICO

COORDINACIÓN GENERAL DE PREGRADO

INGENIERÍA INFORMÁTICA

ASIGNATURA: INGENIERÍA DEL SOFTWARE II

SECCIÓN 1

“Gestión de Configuración, Versionamiento y DevOps”

DOCENTE:

Félix Márquez

ALUMNOS:

Luis Millán

V - 30.040.213

Endrys Flores

V - 30.451.556

Robert Castro

V - 30.994.049

Jesus Báez

V - 26.753.871

Ciudad Guayana, Septiembre del 2026.

ÍNDICE.

INTRODUCCIÓN	3
Ejercicio 1. Flujo de Trabajo en Git y Pull Requests (Control de Cambios Moderno).	4
Ejercicio 2. Gestión de Entornos e Infraestructura como Código.	11
Reflexión: Trazabilidad y Auditoría de la Infraestructura como Código (IaC) vs. Registros Manuales.	14
Pregunta 2 (Análisis de Modelos de Ramificación)	15
Pregunta 3: Evolución del Control de Cambios	17
Pregunta 4 (Artefactos y Auditoría).	19
CONCLUSIONES	22
REFERENCIAS	24

INTRODUCCIÓN

En el mundo actual de la Ingeniería de Software, ha habido un cambio muy grande desde los métodos antiguos de Gestión de Configuración de Software. Hoy en día, las organizaciones trabajan con enfoques como DevOps y la filosofía de Everything as Code. Esto ha transformado cómo se entrega, se evalúa y se mantiene la calidad del software.

Antes, los procesos dependían mucho de revisiones manuales, aprobaciones lentas y documentos que no se actualizaban. Ahora, todo se hace mediante flujos automatizados, integraciones continuas y una infraestructura que se define con código y se almacena en repositorios. Esto permite un control más preciso y una entrega más rápida. Este informe analiza tanto los aspectos teóricos como los prácticos de estos cambios en el ciclo de desarrollo de software. Se centra en un sistema SaaS de facturación, donde se gestionan nuevas características y errores urgentes en producción al mismo tiempo. Se usa el modelo GitHub Flow, que permite controlar los cambios de forma clara y organizada. Se emplean plantillas de Pull Request para estandarizar las modificaciones. También se utilizan Quality Gates en la canalización de Integración Continua para asegurar que solo el código de calidad pase a producción. Además, se hace uso de Docker para contenerizar las aplicaciones, lo que evita problemas de configuración entre diferentes entornos. El informe también analiza el modelo Trunk-Based Development, explicando por qué es eficiente, basándose en los resultados del estudio Accelerate. Este enfoque promueve la entrega frecuente y la colaboración en un solo ramo principal.

Por último, se revisa cómo la Junta de Control de Cambios (CCB), que antes era un grupo de personas que aprobaban cambios, ahora se transforma en auditorías automáticas y seguras basadas en los registros de Git. Esto hace que el control de cambios sea más transparente, trazable y confiable.

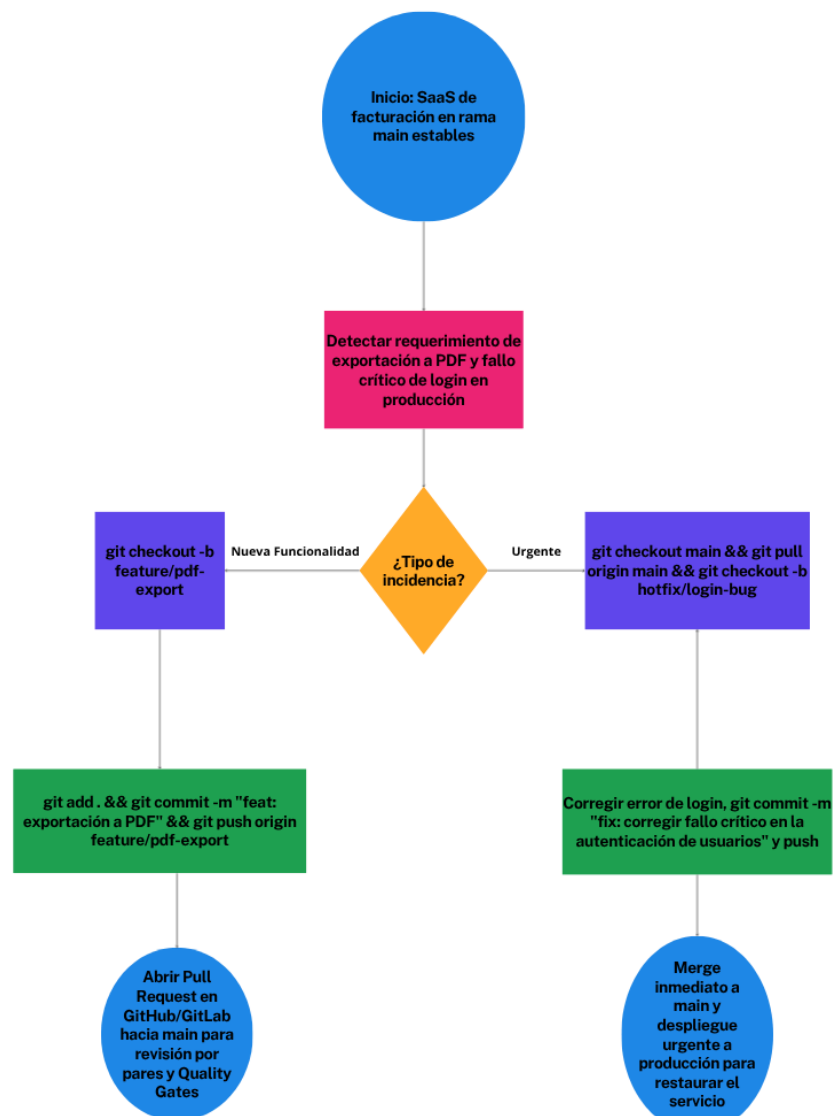
Ejercicio 1. Flujo de Trabajo en Git y Pull Requests (Control de Cambios Moderno).

- **Contexto:** Usted es el líder técnico de un equipo que desarrolla un SaaS de facturación.

Un desarrollador ha terminado una nueva funcionalidad de "exportación a PDF" en su rama local feature/pdf-export. Al mismo tiempo, se detecta un error crítico en producción que impide que los usuarios inicien sesión (bug en la rama main).

1. Diagrama de Flujo.

Flowchart



2. Secuencia Exacta de Comandos de Git y Acciones en Plataforma

Bajo el modelo GitHub Flow (donde main representa siempre un estado estable desplegable a producción), la gestión en paralelo del desarrollo de la funcionalidad feature/pdf-export y la resolución del fallo crítico de autenticación (hotfix/login-bug) se realiza mediante el siguiente flujo secuencial:

Fase A: Publicación segura de la funcionalidad (feature/pdf-export).

1) Creación de la rama de característica local e inicio del desarrollo:

```
git checkout main
```

```
git pull origin main
```

```
git checkout -b feature/pdf-export
```

2) Registro de cambios y envío al repositorio remoto:

```
git add
```

```
git commit -m "feat: agregar módulo de exportación de facturas a PDF"
```

```
git push origin feature/pdf-export
```

3) Acción en la plataforma (GitHub/GitLab):

- Abrir un Pull Request (PR) desde feature/pdf-export hacia main.
- Asignar revisores para la evaluación por pares (Peer Review).
- La fusión permanecerá bloqueada hasta la aprobación manual de los revisores y la validación exitosa de los Quality Gates del pipeline de CI.

Fase B: Atención e integración urgente del Hotfix de Producción

a) Aislamiento del entorno de desarrollo y actualización desde main:

Asegurar cambios de la característica si están pendientes sin guardar

git stash

git checkout main

git pull origin main

b) Creación e implementación de la rama de Hotfix:

git checkout -b hotfix/login-bug

... aplicación de la corrección del código de inicio de sesión ...

git add .

git commit -m "fix: corregir fallo crítico en la autenticación de usuarios"

git push origin hotfix/login-bug

c) Acción en la plataforma (GitHub/GitLab) e integración urgente:

- Abrir un Pull Request (PR) urgente desde hotfix/login-bug hacia main.
- Tras superar los tests del pipeline de CI en tiempo récord, realizar el Merge a main.
- Desplegar a producción inmediatamente para restaurar la disponibilidad del servicio.

d) Sincronización del desarrollador en la rama de la característica:

git checkout feature/pdf-export

Traer el hotfix actualizado desde main para evitar conflictos posteriores

git fetch origin

git rebase origin/main

O en su defecto: git merge origin/main

Plantilla de Pull Request: “Exportación a PDF”

La siguiente plantilla se define como archivo de configuración versionado del repositorio (por ejemplo `.github/PULL_REQUEST_TEMPLATE.md`), de modo que se aplique automáticamente cada vez que un desarrollador abra un Pull Request. Esto convierte al PR en el primer filtro de calidad del proceso, exigiendo que la lógica del cambio, su impacto y su nivel de pruebas queden documentados antes de que un revisor humano o el pipeline de CI intervengan.

Encabezado

Título: [PDF-Export] feat: Exportación de facturas a PDF

Rama origen → destino: feature/pdf-export → main

Tipo de cambio: Nueva funcionalidad (feature)

Issue relacionado: Cierra #<número de ticket>

1. Descripción

Plantilla a completar por el autor del PR:

Descripción

¿Qué hace este cambio?

Implementa la exportación de facturas del módulo de facturación al formato PDF, mediante un nuevo endpoint y un botón en el panel del usuario.

¿Por qué es necesario?

Los clientes solicitan un comprobante descargable de sus facturas

para fines contables y de auditoría.

¿Cómo se implementó? (resumen técnico)

- Nueva librería de generación de PDF: <nombre y versión>
- Nuevo endpoint: GET /api/invoices/{id}/pdf
- Nuevo componente de UI: BotonExportarPDF

Capturas / evidencia:

<adjuntar captura o GIF del PDF generado>

2. Impacto

Plantilla a completar por el autor del PR:

Impacto

Módulos / servicios afectados:

☐ Facturación ☐ API pública ☐ Base de datos ☐ UI

¿Introduce cambios incompatibles (breaking changes)?

Sí / No — detallar

¿Requiere migración de base de datos?

Sí / No — adjuntar script de migración versionado

Impacto en rendimiento:

Tiempo estimado de generación de un PDF, tamaño de la dependencia agregada y efecto en el tiempo de build.

Nivel de riesgo: Bajo / Medio / Alto

Plan de reversión (rollback):

Se despliega detrás de un feature flag 'pdf_export_enabled' para permitir desactivación inmediata sin necesidad de revertir el despliegue.

3. Checklist de revisión

Casillas obligatorias que el autor y el revisor deben verificar antes de aprobar el PR:

- ☐ ¿Se agregaron pruebas unitarias que cubren la nueva funcionalidad?
- ☐ ¿Se revisó la seguridad de la librería de PDF utilizada (vulnerabilidades conocidas / CVEs y licencia)?
- ☐ ¿El código pasa el linter y el análisis estático (SAST) sin hallazgos críticos?
- ☐ ¿Se probó manualmente el flujo completo en un entorno de staging?
- ☐ ¿La funcionalidad está protegida por un feature flag para despliegue seguro?
- ☐ ¿Se actualizó la documentación técnica o el README correspondiente?
- ☐ ¿Se verificó que no se expongan secretos, llaves o credenciales en el código?
- ☐ ¿El pipeline de CI (build, pruebas y Quality Gates) finalizó en estado exitoso?

Esta plantilla operacionaliza el principio de SCM de que ningún ítem de configuración se modifica sin un procedimiento de control: aquí el procedimiento ya no depende de una reunión de comité,

sino de un formulario estructurado y verificable que precede a la revisión por pares y a la validación automática del pipeline de CI.

Quality Gates del Pipeline de CI

Antes de que el botón de “Merge” quede habilitado sobre la rama main, el pipeline de CI (por ejemplo GitHub Actions o GitLab CI) debe ejecutar y aprobar automáticamente las siguientes tres puertas de calidad. Si cualquiera de ellas falla, el PR queda bloqueado hasta que el problema se corrija, sin intervención manual de un comité.

1	Pruebas automatizadas y cobertura mínima de código El pipeline ejecuta la batería completa de pruebas unitarias y de integración sobre la rama del PR. El Merge solo se habilita si el 100% de las pruebas pasa y la cobertura de código no disminuye por debajo de un umbral definido (por ejemplo, 80%). • Build y suite de pruebas en estado “verde” (sin fallos). • Cobertura de código $\geq 80\%$ en los módulos modificados. • Prueba específica de generación de PDF (contenido, tamaño y formato del archivo resultante).
----------	--

2	Análisis estático y de seguridad (SAST + SCA) El pipeline ejecuta análisis estático de código (SAST) sobre el nuevo código y un Análisis de Composición de Software (SCA) sobre las dependencias, incluyendo la librería de generación de PDF, para detectar vulnerabilidades conocidas (CVEs) y violaciones de licencia. • Cero vulnerabilidades de severidad crítica o alta en dependencias (SCA). • Sin hallazgos críticos de SAST ni secretos/credenciales expuestos en el código.
----------	---

	• Licencia de la librería de PDF compatible con la política de la organización.
--	---

3	Aprobación de revisión por pares (Peer Review) El sistema de control de versiones exige al menos una aprobación explícita de un revisor distinto al autor, con todos los comentarios bloqueantes resueltos y la rama actualizada respecto a main, antes de permitir la fusión. • Mínimo una aprobación (“approve”) de un revisor con permisos sobre el repositorio. • Cero comentarios o “change requests” abiertos sin resolver. • Rama sin conflictos y sincronizada con el último estado de main.
----------	---

En conjunto, estas tres puertas reproducen digitalmente las funciones que antes cumplía una CCB (verificación técnica, verificación de seguridad y aprobación de un tercero), pero de forma consistente, instantánea y aplicada a cada cambio sin excepción.

Ejercicio 2. Gestión de Entornos e Infraestructura como Código.

1. Dockerización de la Aplicación Node.js

Para dar solución a los problemas de configuración en el entorno de despliegue, se diseñó un archivo de definición de contenedor (Dockerfile) aplicando la técnica avanzada de multi-stage builds (construcción en múltiples etapas). Este enfoque permite desacoplar el entorno de compilación e instalación de dependencias del entorno final de ejecución, garantizando imágenes ligeras y seguras.

```

|# =====
# ETAPA 1: Construcción e Instalación (Build)
# =====
# Usamos una imagen oficial de Node.js ligera basada en Alpine Linux
FROM node:20-alpine AS builder

# Definimos el directorio de trabajo dentro del contenedor
WORKDIR /usr/src/app

# Copiamos los archivos de gestión de dependencias
COPY package*.json ./

# Instalamos únicamente las dependencias de producción de forma limpia
RUN npm ci --only=production

# Copiamos el código fuente de la aplicación
COPY server.js ./

# =====
# ETAPA 2: Ejecución (Production)
# =====
# Partimos de una imagen limpia para reducir la superficie de ataque
FROM node:20-alpine

WORKDIR /usr/src/app

# Copiamos las dependencias y el código desde la etapa anterior
COPY --from=builder /usr/src/app ./

# Buenas prácticas de seguridad: Cambiamos del usuario 'root' a 'node' (non-root)
USER node

# Documentamos que la aplicación escucha en el puerto 8080
EXPOSE 8080

# Definimos la variable de entorno para entorno de producción
ENV NODE_ENV=production

# Comando para iniciar la aplicación Node.js
CMD ["node", "server.js"]

```

2. Mitigación de la Deriva de Configuración (Configuration Drift) y el Síndrome

"Funciona en mi Máquina"

El fenómeno conocido habitualmente como el síndrome "funciona en mi máquina" surge por la divergencia entre los entornos locales de desarrollo y los servidores de prueba o producción. Estas variaciones incluyen discrepancias en las versiones de librerías del sistema operativo, el entorno de ejecución (runtime) de Node.js, parches de seguridad y scripts de configuración ejecutados manualmente sin control de versiones.

El Dockerfile resuelve esta problemática mediante dos principios estructurales de la Ingeniería de Software y DevOps:

- **Paridad de Entornos e Inmutabilidad:** La contenedorización empaqueta la aplicación junto con sus dependencias directas, variables y configuraciones del sistema operativo dentro de un artefacto estandarizado. En lugar de modificar los servidores in situ mediante scripts mutables, se genera una imagen inmutable en la etapa de Integración Continua (CI).
- **Determinismo Operativo:** La misma imagen evaluada en las etapas de prueba es la que se despliega en producción. Al eliminar la intervención manual sobre la infraestructura, se erradica la deriva de configuración (configuration drift), asegurando que el comportamiento del software sea idéntico independientemente del entorno físico o virtual subyacente.

Artefactos de SCM Modernos y Control de Ítems de Configuración

En la SCM clásica (según estándares como IEEE 828 o SWEBOK), el "Plan de Gestión de Configuraciones" era un documento de texto o Word de carácter puramente descriptivo y burocrático. En la era DevOps y bajo la filosofía de "Everything as Code", la gestión de configuración deja de ser un documento pasivo y se materializa en archivos de configuración declarativos, versionados directamente dentro del repositorio de código fuente.

Reflexión: Trazabilidad y Auditoría de la Infraestructura como Código (IaC) vs. Registros Manuales.

“¿Qué ventajas de trazabilidad y auditoría (conceptos clásicos de SCM) se obtienen al tratar la infraestructura como código en Git, en comparación con los logs manuales de un administrador de sistemas?”

Desde la perspectiva clásica de la Gestión de Configuración de Software (SCM), la transición hacia la Infraestructura como Código (IaC) en repositorios Git representa un salto cualitativo sobre la administración tradicional de sistemas.

- **Historial Inmutable vs. Registros Volátiles:** Los logs de servidor (como `.bash_history`) y las bitácoras manuales son vulnerables a la omisión humana, la alteración o la pérdida accidental. Por el contrario, la IaC alojada en Git proporciona un registro inmutable y firmado criptográficamente de cada cambio en la infraestructura. Cada modificación queda asociada a un autor, una fecha, una justificación de negocio y un proceso formal de revisión por pares (Pull Request).
- **Gobernanza Automatizada y Línea Base:** En la administración manual, verificar el cumplimiento de políticas de seguridad requiere auditorías forenses tardías. Con IaC, la infraestructura se define como una línea base declarativa. El cumplimiento normativo se valida de forma automatizada mediante pipelines de CI (Quality Gates) y herramientas de análisis estático, lo que garantiza la reproducibilidad y trazabilidad continua del sistema.

Pregunta 2 (Análisis de Modelos de Ramificación)

“Compare rigurosamente los modelos GitFlow y Trunk-Based Development. Utilizando los conceptos del libro Accelerate (Forsgren et al.), argumente por qué la industria de alto rendimiento se está moviendo hacia Trunk-Based. ¿Qué prácticas de SCM ejemplo CI automatizado, son obligatorias para que Trunk-Based no resulte en un código inestable en producción?”

Tabla de Análisis Comparativo: GitFlow vs. Trunk-Based Development (TBD).

Criterio de Comparación	GitFlow	Trunk-Based Development (TBD)
Estructura de Ramas	Múltiples ramas de larga vida (main, develop) y ramas efímeras (feature, release, hotfix).	Una única rama principal (main o trunk) con ramas de características extremadamente efímeras (pocas horas).
Frecuencia de Integración	Integración tardía o por ciclos/lanzamientos planificados.	Integración continua, múltiples veces al día por cada desarrollador.
Riesgo en Resoluciones	Alto riesgo de "Merge Hells" (conflictos masivos de fusión al integrar ramas de larga vida)	Riesgo mínimo; la integración constante detecta incompatibilidades de forma temprana.
Mecanismo de Despliegue	Basado en ramas de versiones (<i>release branches</i>) predecibles	Basado en el desacoplamiento mediante Feature Flags (interruptores de funcionalidad)
Ciclos de Feedback	Lentos; las fallas de integración se detectan al final del ciclo de desarrollo.	Rápido y en tiempo real a través de ejecuciones constantes de CI

Argumentación con Base en el Libro Accelerate (Forsgren et al., 2018)

En el estudio de Accelerate (Forsgren, Humble, & Kim, 2018), se demuestra empíricamente que la velocidad y estabilidad del software no son objetivos contradictorios, sino concurrentes en organizaciones de alto rendimiento (High-Performing Teams). Las razones por las cuales la industria de alto rendimiento migra decisivamente hacia Trunk-Based Development son:

- **Reducción del Inventario de Trabajo en Proceso (WIP):** El código guardado en ramas de larga duración durante semanas constituye software no entregado y sin validar. TBD minimiza el WIP permitiendo que el valor fluya continuamente hacia el tronco común.
- **Eliminación del Dolor de Fusión (Merge Pain):** Las fusiones complejas consumen horas de ingeniería no productiva. En TBD, los cambios pequeños facilitan las revisiones y la automatización de pruebas.
- **Desacoplamiento entre Despliegue y Liberación:** Mediante el uso de Feature Flags, el código se despliega en producción en un estado latente/oculto sin afectar a los usuarios. Esto permite realizar pruebas canario (canary releases) o pruebas A/B de bajo riesgo directamente sobre el entorno real.

Prácticas Obligatorias de SCM para Garantizar la Estabilidad en TBD.

Para adoptar Trunk-Based Development sin degradar la estabilidad del entorno de producción, es obligatorio contar con las siguientes disciplinas y salvaguardas de SCM:

- a) **Integración Continua (CI) Automatizada e Inflexible:** Cada commit o push enviado al tronco debe desencadenar un pipeline que compile y ejecute la suite de pruebas unitarias y de integración en cuestión de minutos. Si la compilación o una prueba falla, el pipeline debe marcar inmediatamente el tronco como bloqueado (Broken Build) hasta corregir la regresión.

- b) **Gobernanza de Feature Flags (Feature Toggles):** Implementación estricta de condicionales lógicos para ocultar código incompleto en producción. Se requiere además una gestión activa para remover las banderas obsoletas y evitar la acumulación de deuda técnica en la base de código.
- c) **Pruebas de Cobertura y Bisect Automatizado:** Contar con umbrales mínimos de cobertura de código (Quality Gates) y la capacidad de utilizar técnicas de búsqueda binaria de errores (como git bisect) para revertir commits defectuosos de forma automatizada e inmediata.
- d) **Monitoreo y Observabilidad en Tiempo Real:** Sistemas de telemetría que midan la tasa de errores y latencia inmediatamente después de cada integración para realizar rollbacks instantáneos en caso de anomalías.

Pregunta 3: Evolución del Control de Cambios

“La Junta de Control de Cambios (CCB) tradicional es un cuello de botella que viola los principios de la Entrega Continua”.

La afirmación es válida como diagnóstico del modelo clásico

La CCB tradicional fue diseñada para un contexto de líneas base estables y ciclos de liberación largos, donde un comité humano se reunía periódicamente para aprobar cambios sobre documentación, código o configuración. Este modelo introduce al menos tres fricciones incompatibles con la Entrega Continua: primero, es asíncrono por naturaleza, ya que un cambio debe esperar a la próxima sesión del comité en lugar de integrarse cuando está listo; segundo, es subjetivo y no escalable, porque depende del criterio y la disponibilidad de un grupo reducido de personas que no pueden revisar en profundidad decenas de cambios diarios; y tercero, su trazabilidad es débil, ya que las decisiones suelen quedar registradas en actas o minutas dispersas, difíciles de vincular de forma inequívoca con el cambio técnico

concreto que aprobaron. En un flujo de Trunk-Based Development o GitHub Flow, donde se espera integrar cambios varias veces al día, un comité que se reúne semanalmente se convierte, en efecto, en el paso más lento de todo el proceso de entrega.

El Pull Request y la CI no eliminan el control de cambios: lo automatizan

Es importante matizar la afirmación: la Entrega Continua no elimina el control de cambios, elimina la burocracia manual y centralizada como único mecanismo de control. La función que cumplía la CCB (evaluar si un cambio es seguro, correcto y apto para producción) se conserva, pero se descompone en dos mecanismos automatizados y descentralizados que actúan sobre cada commit:

1. El Pull Request como primer filtro de calidad: sustituye la aprobación colegiada por una revisión por pares (peer review) distribuida entre el propio equipo de desarrollo, que conoce el contexto técnico del cambio mejor que un comité externo. El PR no solo evalúa la lógica del código, sino también su impacto arquitectónico y de seguridad, y deja un registro permanente y vinculado al commit exacto que se está aprobando.
2. El pipeline de Integración Continua como puerta técnica objetiva: cada Pull Request dispara automáticamente la ejecución de pruebas unitarias, análisis estático (SAST), escaneo de dependencias (SCA) y verificación de cobertura. Estos Quality Gates aplican el mismo criterio, sin excepciones ni fatiga humana, a cada cambio, bloqueando el Merge de forma inmediata si no se cumplen los umbrales definidos.

Por qué este modelo es más seguro, rápido y auditable

En términos de seguridad, el control automatizado aplica verificaciones de seguridad (SAST, SCA, escaneo de secretos) al 100% de los cambios y en cada iteración, algo que una CCB humana, limitada por tiempo y capacidad de atención, no puede sostener de forma consistente; este es precisamente el principio de Shift-Left Security que plantea la unidad. En términos de velocidad, el feedback pasa de ciclos de días o semanas (esperar la próxima reunión del comité) a minutos, ya que el

pipeline responde de forma inmediata al abrir el PR, lo cual es coherente con los hallazgos del libro Accelerate: los equipos de alto rendimiento despliegan con mayor frecuencia y se recuperan más rápido de fallos precisamente porque automatizan estas validaciones. En términos de auditabilidad, cada Merge queda respaldado por evidencia inmutable y verificable en el propio repositorio Git: el historial de commits, el autor y los revisores del PR, los comentarios de la revisión y el resultado exacto del pipeline de CI (qué pruebas corrieron, con qué resultado y en qué momento) sustituyen a un acta de reunión, ofreciendo una traza mucho más precisa, granular y difícil de manipular que un registro manual.

Pregunta 4 (Artefactos y Auditoría).

“En la SCM clásica, el "Plan de Gestión de Configuraciones" era un documento de Word. En la era DevOps, este plan se materializa en archivos de texto dentro del repositorio. Identifique y explique la función de al menos 4 archivos de configuración tales como .gitignore, Dockerfile, .github/workflows/ci.yml y cómo cada uno garantiza la integridad, reproducibilidad y control de un ítem de configuración moderno.”

A continuación, se identifican y explican cuatro (4) archivos de configuración fundamentales que operan como Ítems de Configuración (CI) modernos, garantizando la integridad, reproducibilidad y control del software:

1. `.gitignore` (Gobernanza y Limpieza del Repositorio)

- **Función:** Define patrones de archivos y directorios explícitos que deben ser ignorados por el sistema de control de versiones Git.

- **Garantía de SCM:** Garantiza la integridad y seguridad del repositorio impidiendo que artefactos temporales de compilación (`node\_modules/`, `dist/`), archivos de entorno local con

credenciales sensibles (`.env`, certificados) o logs del sistema operativo se suban accidentalmente. De este modo, preserva la línea base del proyecto limpia, ligera y auditable.

2. `Dockerfile` (Inmutabilidad y Paridad del Entorno)

- **Función:** Especifica de forma declarativa e imperativa los pasos necesarios para construir la imagen del contenedor del sistema (runtime, sistema base, librerías y dependencias).

- **Garantía de SCM:** Elimina la deriva de configuración (configuration drift) y el síndrome "funciona en mi máquina". Convierte al entorno de ejecución completo en un Ítem de Configuración versionado en Git, garantizando la reproducibilidad exacta del entorno entre las fases de desarrollo, pruebas y producción.

3. `.github/workflows/ci.yml` (Pipeline de Control de Cambios Automatizado)

- **Función:** Define las etapas, eventos desencadenantes (triggers) y Quality Gates del pipeline de Integración Continua en GitHub Actions.

- **Garantía de SCM:** Reemplaza el control manual de cambios por reglas de validación técnica automatizadas. Garantiza la calidad e integridad del código obligando a que cada commit o Pull Request ejecute pruebas unitarias, análisis SAST/SCA y linters antes de permitir la fusión con la rama principal (main).

4. `package.json` / `package-lock.json` (Gestión Determinista de Dependencias)

- **Función:** Declara las dependencias de código fuente de terceros (módulos de Node.js) y sus versiones exactas (vía árbol de hashes criptográficos en el lockfile).

- **Garantía de SCM:** Asegura la reproducibilidad determinista de las compilaciones. Al fijar las versiones exactas y los hashes de las librerías, se evita que actualizaciones de terceros rompan

el código en producción o introduzcan vulnerabilidades no rastreadas en la cadena de suministro de software (Software Supply Chain).

CONCLUSIONES

La evolución de la Gestión de Configuración de Software (SCM) hacia el paradigma de DevOps y "Everything as Code" ha redefinido drásticamente la forma en que los equipos de ingeniería garantizan la calidad, seguridad y velocidad de entrega del software.

A través del desarrollo de esta unidad y la resolución de las problemáticas planteadas, se han consolidado los siguientes hallazgos fundamentales:

1. **La Automatización como Sustituto de la Burocracia:** La sustitución de la Junta de Control de Cambios (CCB) tradicional por flujos modernos basados en Pull Requests y pipelines de CI/CD automatizados demuestra que la velocidad y la estabilidad no son contradictorias. La inclusión de Quality Gates (pruebas automatizadas, análisis SAST/SCA y revisiones por pares) ofrece un control de cambios más riguroso, instantáneo e inmutable que cualquier revisión manual.
2. **Adopción de Trunk-Based Development:** Apoyados en los principios del libro **Accelerate**, queda evidenciado que las organizaciones de alto rendimiento minimizan el trabajo en proceso (WIP) y los conflictos de fusión mediante ramas cortas y la integración continua al tronco (main), utilizando Feature Flags para desacoplar el despliegue del código de su liberación a producción.
3. **Erradicación de la Deriva de Configuración:** La Infraestructura como Código (IaC) y la contenedorización con Docker resuelven de forma estructural el problema de "funciona en mi máquina". Al tratar los entornos de ejecución, las dependencias y las canalizaciones de despliegue como archivos de texto versionados en Git, se garantiza la

reproducibilidad determinista, la paridad de entornos y la completa auditabilidad forense del ciclo de vida del software.

En conclusión, la SCM moderna no busca frenar el cambio mediante procesos lentos, sino habilitar un flujo continuo de entregas de alto valor con riesgos calculados, trazabilidad total y salvaguardas automatizadas en cada etapa del desarrollo.

REFERENCIAS

Fowler, M. (2013, 15 de mayo). Feature toggles. Martinfowler.com.

<https://martinfowler.com/articles/feature-toggles.html>

Fowler, M. (2020, 10 de mayo). Trunk-based development. Martinfowler.com.

<https://martinfowler.com/articles/branching-patterns.html#Trunk-basedDevelopment>

GitHub. (s. f.). GitHub flow. GitHub Docs. Recuperado el 22 de septiembre de 2026, de

<https://docs.github.com/en/get-started/using-github/github-flow>

IEEE Computer Society. (2012). IEEE standard for configuration management in systems and software engineering (IEEE Std 828-2012). Institute of Electrical and Electronics Engineers.

<https://doi.org/10.1109/IEEESTD.2012.6170935>

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4.^a ed.).

Addison-Wesley.

Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The science of lean software and DevOps: Building and scaling high performing technology organizations*. IT Revolution Press.

Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

IEEE Computer Society. (2014). *Guide to the Software Engineering Body of Knowledge (SWEBOK V3/V4)*. IEEE.